Lecture 21: Post Office Mail Protocol, POP3 (RFC 1939)

Objectives:

- Learn about the POP protocol
- Lean how to write a POP client to retrieve mails

1. Overview of POP protocol

The objective of the **Post Office Protocol** (**POP**) is to allow users to retrieve their mails from their mail servers to their local machines.

POP provides a way of saving disk space on the mail server and relieving the network admin from managing the mails. However, it may not be very convenient for users accessing their mails through multiple machines.

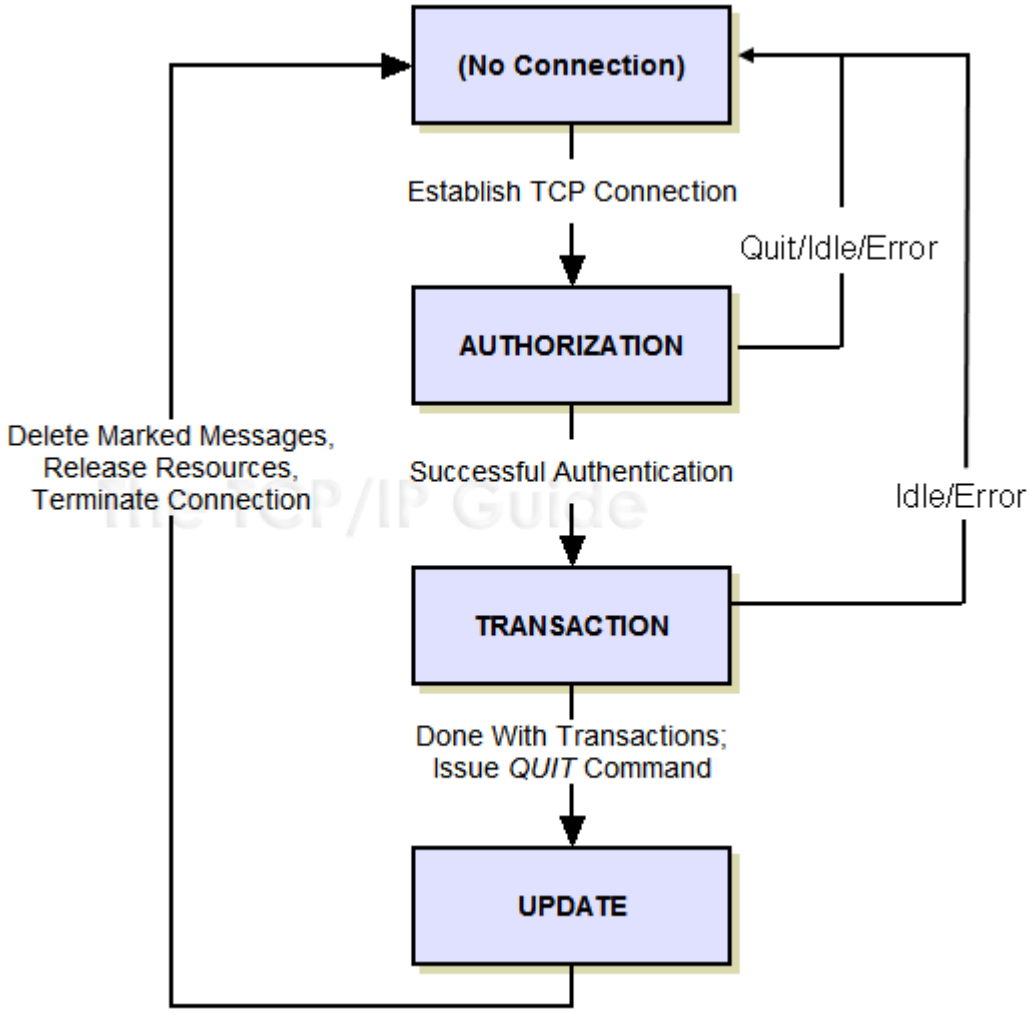
The alternative is **Internet Mail Access Protocol** (**IMAP) –**RFC 1730, which allows users to manipulate their mails directly on the mail server.

The POP server runs on a well-known port **110**.

The protocol provides few commands that the client can use to communicate with the server.

For each command, the server responds with one of two responses called status indicators; **"+OK"** followed by some textual comments if the commands succeeds, or **"-ERR"** followed by textual comments if the command fails.

POP is a **stateful** protocol with three states as summarized below. At each state, only a subset of the commands can be issued.



No Connection:

The client first establishes a TCP connection with the POP server on port **110**.

After a connection request succeeds, the server sends a welcome message to the client and enters into the Authorization state.

Authorization State:

In this state, the client either sends a "**QUIT**" command to terminate the connection or issues a "**USER username**" command, followed by "**PASS password**" command to login.

Once the login is successful, the server acquires an exclusive access lock on the maildrop, to prevent messages from being modified or removed before the session enters the UPDATE state.

If the lock is successfully acquired, the server responds with a positive status indicator.

If the maildrop cannot be opened for some reason (for example, a lock can not be acquired, the client is denied access.

After the POP3 server has opened the maildrop, it assigns a message number to each message, and notes the size of each message in bytes.

The first message in the maildrop is assigned a message-number of "1", the second is assigned "2", and so on.

The server then enters into the Transaction State.

Transaction State:

In the transaction state, the user can issue any of the following transaction commands repeatedly until he enters the **QUIT** command, which puts the server into the **UPDATE** state.

Command	Responses	Examples
QUIT	+OK	+OK Server closing connection
STAT	+OK nn mm	STAT +OK 2 320
LIST [msg]	+OK scan listing follows -ERR no such message	LIST +OK 2 messages (320 octets) 1 120 2 200 . ... LIST 2 +OK 2 200
RETR msg	+OK message follows -ERR no such message	RETR 1 +OK 120 octets < the POP3 server sends the entire message here >
DELE msg	+OK message deleted -ERR no such message	DELE 2 +OK message deleted
NOOP	+OK no transaction	NOOP

		+OK
RSET	+OK	RSET +OK maildrop has 2 messages (320 octets)
TOP msg nn	+OK top of msg -ERR	TOP 1 10 +OK < first 10 lines of the header >

Notes:
For multi-line responses, such as responses from LIST, RETR and TOP commands, the response lines ends with "." on a line by itself.

If a message is marked for deletion with a DELE command, then its message number is not a valid argument for any command. Also the message will not be shown as part of a response.

While still in the Transaction State, messages marked for deletion can be unmarked by issuing a RSET command.

Update State:
When the client issues the QUIT command from the TRANSACTION state, the server enters the UPDATE state.

In this state, the server removes any message marked for deletion from the maildrop, then releases the exclusive-access lock on the maildrop and closes the TCP connection.

If a session terminates for some reason other than a client-issued QUIT command, the server does NOT enter the UPDATE state and MUST not remove any messages from the maildrop.

The following is a sample POP3 conversation using telnet.

```
telnet bareed.ccse.kfupm.edu.sa 110
+OK POP3 khuzama v2000.69 server ready
user bmghandi
+OK User name accepted, password please
pass actual_password
+OK Mailbox open, 24 messages
stat
+OK 24 3535201
list
+OK Mailbox scan listing follows
1 355411
2 3733
...//deleted
24 4590
.
top 10 5
+OK Top of message follows
Received: from khuzama.ccse.kfupm.edu.sa (localhost
[127.0.0.1])
        by khuzama.ccse.kfupm.edu.sa (8.11.0/8.9.3) with
ESMTP id
...//deleted
To: <bmghandi@ccse.kfupm.edu.sa>
Subject: SWE344 - Project proposal...
Message-ID:
<1070190821.3fc9d0e5c948f@webmail.ccse.kfupm.edu.sa>
Date: Sun, 30 Nov 2003 14:13:41 +0300
From: "Al-Tawfiq, Hani" <st204920@ccse.kfupm.edu.sa>
MIME-Version: 1.0
Content-Transfer-Encoding: 8bit
User-Agent: IMP/PHP IMAP webmail program 2.2.0-pre13
X-Originating-IP: 212.93.193.82
Content-Type: text/plain
Content-Length: 838
Status: RO

Assalam Alaikum
Dear Mr. Ghandi

Eid Mubarak to you. I hope you enjoyed the vacation and I
hope you are in good health
.
```

2. Writing a simple POP Client

The following example shows how to write a simple POP client in C#

```
using System;
using System.Windows.Forms;
using System.Net;
using System.Net.Sockets;
using System.Threading;
using System.IO;

namespace MailApplications
{
  class SimpleMailReader : System.Windows.Forms.Form
  {
    private System.Windows.Forms.Label label3;
    private System.Windows.Forms.Label label2;
    private System.Windows.Forms.Button btClose;
    private System.Windows.Forms.TextBox password;
    private System.Windows.Forms.TextBox hostname;
    private System.Windows.Forms.Label label4;
    private System.Windows.Forms.Label label;
    private System.Windows.Forms.TextBox status;
    private System.Windows.Forms.TextBox username;
    private System.Windows.Forms.ListBox messages;
    private System.Windows.Forms.Button btLogin;

    private TcpClient client;
    private NetworkStream ns;
    private StreamReader reader;
    private StreamWriter writer;

    public SimpleMailReader()
    {
      InitializeComponent();
    }

    void btLoginClick(object sender, System.EventArgs e)
    {
      status.Text = "Checking for messages....";
      Thread connectionHandler = new Thread(new
ThreadStart(HandleConnection));
      connectionHandler.IsBackground = true;
      connectionHandler.Start();
```

```
    }

    void HandleConnection() {
      string response;
      string from = null;
      string subject = null;
      int messageCount;

      try {
        client = new TcpClient(hostname.Text, 110);
      }
      catch(SocketException) {
        status.Text = "Unable to connect to server";
      }

      ns = client.GetStream();
      reader = new StreamReader(ns);
      writer = new StreamWriter(ns);

      response = reader.ReadLine(); //get welcome message
      if (response.StartsWith("-ER")){
        status.Text = "Unable to connect to server";
        return;
      }

      IssueCommand("User "+username.Text);
      IssueCommand("Pass "+password.Text);
      response = IssueCommand("Stat");

      string[] tokens = response.Split(' ');
      messageCount = int.Parse(tokens[1]);
      if (messageCount > 0)
        status.Text = "You have " + messageCount+ "
messages";
      else
        status.Text = " You have no messages";

      for (int i = 1; i<=messageCount; i++) {
        IssueCommand("top "+ i + " 20");

        while (true) {
          response = reader.ReadLine();
          if (response == ".")
            break;
          if (response.ToLower().StartsWith("from"))
```

```
            from = response;
          else if (response.ToLower().StartsWith("subject"))
            subject = response;

        }

        messages.Items.Add(i + " "+ from + " "+ subject);
      }

    }

    string IssueCommand(string command) {
      string response = null;
      try {
        writer.WriteLine(command);
        writer.Flush();

        response = reader.ReadLine();
        if (response.StartsWith("-ER"))
          status.Text =  "Unable to connect to server";
      }
      catch(SocketException) {
        status.Text = "Unable to connect to server";
      }

      return response;
    }

    void btCloseClick(object sender, System.EventArgs e)
    {
      if (ns != null) {
        reader.Close();
        writer.Close();
        ns.Close();
        client.Close();
      }
      Close();
    }

    void OnGetMessage(object sender, System.EventArgs e) {
      string title = (string) messages.SelectedItem;
      ShowMessage sm = new ShowMessage(ns, title);
      sm.ShowDialog();
    }
```

```
    void InitializeComponent() {
      //deleted
    }

    [STAThread]
    public static void Main(string[] args)
    {
      Application.Run(new SimpleMailReader());
    }
  }
}
```

```
using System;
using System.Windows.Forms;
using System.Net;
using System.Net.Sockets;
using System.IO;
using System.Threading;

public class ShowMessage : System.Windows.Forms.Form {
  private System.Windows.Forms.TextBox display;

  private NetworkStream ns;
  private StreamReader reader;
  private StreamWriter writer;
  private Thread handler = null;
  private int messageNumber = 0;

  public ShowMessage(NetworkStream ns, string title){
    InitializeComponent();
    this.ns = ns;
    reader = new StreamReader(ns);
    writer = new StreamWriter(ns);
    Text = title;
    string[] tokens = title.Split(' ');
    messageNumber = int.Parse(tokens[0]);

    handler = new Thread(new ThreadStart(ConnectionHandler));
    handler.Start();
  }


  void ConnectionHandler() {
```

```
    writer.WriteLine("retr "+messageNumber);
    writer.Flush();
    string response = reader.ReadLine();

    int lineCount = 0;

    while (true) {
      response = reader.ReadLine();
      lineCount ++;
      if (response == ".")
        break;
      //display only first 100 lines
      if (lineCount < 100)
        display.Text += response + "\r\n";
    }
  }

  void InitializeComponent() {
    //deleted
  }
}
```